

Problem Description:

A store owner wants to calculate the **total cost, discount, and final bill** for a customer. The store offers a **10% discount** if the total purchase exceeds 500 units. Write an R program that takes the **price of an item and quantity purchased** as input and calculates:

- Total cost
- Discount (if applicable)
- Final bill amount.

Objective:

To evaluate student marks and assign grades using if-else, and switch statements in R, demonstrating how conditional decision-making can be applied to determine grades and provide appropriate feedback.

Sample Input	Expected Output
Enter price of item: 250	Total cost: 750 Discount: 75
Enter quantity purchased: 3	Final bill amount: 675
Enter price of item: 100	Total cost: 500 Discount: 0
Enter quantity purchased: 5	Final bill amount: 500

Program:

```
# Billing System Calculation in R
# User input price and quantity
price <- as.numeric(readline("Enter price of item: "))
quantity <- as.numeric(readline("Enter quantity purchased: "))

# Calculate total cost
total_cost <- price * quantity

# Calculate discount
if (total_cost > 500) {
  discount <- total_cost * 0.10
} else {
  discount <- 0
}
```

Calculate final bill

$\text{final_bill} \leftarrow \text{total_cost} - \text{discount}$

Display results

`cat ("Total cost : ", total_cost, "\n")`

`cat ("Discount : ", discount, "\n")`

`cat ("Final Bill amount: ", final_bill, "\n")`

Result:

Thus, the program correctly calculates the total cost, discount, and final bill using arithmetic operations

Grade Evaluation and Feedback Using Control Statements

Problem Description:

A teacher wants to evaluate students' marks and assign grades based on their score:

- Marks 90 and above → Grade A
- Marks 75 to 89 → Grade B
- Marks 50 to 74 → Grade C
- Marks below 50 → Grade F

Additionally, the program should provide **feedback** for each grade:

- A → "Excellent performance"
- B → "Good performance"
- C → "Average performance"
- F → "Needs improvement"

Write an R program that takes the **marks of a student** as input, assigns the **appropriate grade** using **if** and **if-else**, and provides **feedback** using a **switch** statement.

Objective:

To evaluate student marks and assign grades with corresponding feedback using **if**, **if-else**, and **switch** statements in R, demonstrating conditional decision-making in a practical scenario.

Sample Input	Expected Output
Enter student marks: 80	Grade assigned: B Feedback: Good performance
Enter student marks: 50	Grade assigned: C Feedback: Average performance

Program:

```
# Grade Evaluation Using Control Statements
# Take input marks
```

```
marks <- as.numeric(readline("Enter student marks: "))
```

```
# Using if-else to assign grade
```

```
if (marks >= 90) {
```

```
  grade <- "A"
```

```
} else if (marks >= 75) {
```

```
  grade <- "B"
```

```
} else if (marks >= 50) {
```

```
  grade <- "C"
```

```
} else {
```

Input 8:

Enter student marks: 50

Output 8:

Grade assigned: C

Feedback: Average Performance.

Output 1:

Grade assigned: B

Feedback: Good Performance

Input 1:

Enter student marks: 80

grade ← "F"
?

Display grade

```
cat("grade assigned :", grade, "\n")
```

Using switch statement for feedback

```
feedback ← switch(grade,  
  "A" = "Excellent Performance",  
  "B" = "Good performance",  
  "C" = "Average Performance",  
  "F" = "Needs improvement")  
cat("Feedback: ", feedback, "\n").
```

Result:

Thus, the program assigns grades accurately and provides corresponding feedback using control statements (if, if-else, switch) in R

Function:

```
generate fib fib ← function(n) {  
  fib ← new Array(n)  
  fib[0] ← 0  
  if (n > 1) {  
    fib[1] ← 1  
  }  
  for (i in 3:n) {  
    fib[i] ← fib[i-1] + fib[i-2]  
  }  
  return (fib)  
}
```

Input:

Enter the number of rows: 6

Output:

Fibonacci staircase using for loop:

```
0  
0 1  
0 1 1  
0 1 1 2  
0 1 1 2 3  
0 1 1 2 3 5
```

Program:

```
# Fibonacci Staircase Pattern Using Loops  
# Function to generate nth Fibonacci number
```

```
n ← as.integer(readline(prompt = "Enter the number of  
rows: "))  
fib ← numeric(n)  
fib[1] ← 0  
if (n > 1) {  
  fib[2] ← 1  
}
```

```
# User input
```

```
n ← as.integer(readline(prompt = "Enter the number  
of rows: "))
```

```
# Using for loop
```

```
for (i in 3:n) {  
  fib[i] ← fib[i-1] + fib[i-2]  
}  
cat("Fibonacci Staircase using for loop: \n")  
for (i in 1:n) {  
  for (j in 1:i) {  
    cat(fib[j], " ")  
  }  
  cat("\n")  
}
```


Input:

Enter the number of rows: 6

Output:

Fibonacci Staircase using ^{while} loop:

```
0
0 1
0 1 1
0 1 1 1
0 1 1 1 1
0 1 1 1 1 1
```

Using while loop

```
i ← 3
while (i ≤ n) {
    fib[i] ← fib[i-1] + fib[i-2]
    i ← i + 1
}
```

cat ("Fibonacci Staircase using while loop: \n")

row ← 1

while (row ≤ n) {

col ← 1

while (col ≤ row) {

cat (fib[col], " ")

col ← col + 1

}

cat ("\n")

row ← row + 1

}

Result:

Thus, the program generates a Fibonacci staircase pattern for the given number of rows using both for and while loops. It demonstrates iterative computation of Fibonacci numbers combined with nested loops for pattern printing.

Input:

Enter password : Pass@123

Output:

Password Length : 8

Uppercase Version : PASS@123

Containing Number : TRUE

Containing Special

Character : TRUE

Input 2:

Enter password : Pass

Output:

Password Length : 4

Uppercase Version : PASS

Containing Number : FALSE

Containing Special Character : FALSE

Ex.No: 1D	Password Strength Analysis Using String Operations
Date: 5/2/26	

Problem Description:

An online system requires users to create secure passwords. An R program is needed to analyze a user-entered password by checking its length, converting case, and identifying whether it contains numbers and special characters using string operations.

Objective:

To demonstrate the application of string manipulation functions in R for analyzing and validating password strength.

Sample Input	Expected Output
Enter password: Pass@123	Password Length: 8 Uppercase Version: PASS@123 Contains Number: TRUE Contains Special Character: TRUE
Enter password: Pass	Password Length: 4 Uppercase Version: PASS Contains Number: FALSE Contains Special Character: FALSE

Program:

```
# Password strength analysis using strings in R
```

```
# User input password
```

```
password <- readline(prompt = "Enter password: ")
```

```
# Length of password
```

```
password_length <- nchar(password)
```

```
# Case conversion
```

```
up_password <- toupper(password)
```

```
# Check for number
```

```
contains_number <- grepl("[0-9]", password)
```

```
grepl("[A-Za-z0-9]", password)
```

Check for special character

```
cat("password length: ", password-length, "\n")
```

```
cat("uppercase version: ", up-password, "\n")
```

```
cat("contains Number: ", contains-number, "\n")
```

```
cat("contains Special character: ", contains-special, "\n")
```

Result:

Thus the program analyzes the password using string functions in R to determine its length and character composition. It demonstrates how string operations help in basic password validation.

Ex: No: 1E
Date: 29/1/20

Monthly Sales Analysis Using Vectors

Problem Description:

A retail store records its monthly sales figures in a vector. Write an R program to analyze the sales data by calculating the total sales, average monthly sales, highest sales value, lowest sales value, and identifying months with sales above the average.

Objective:

To apply vector operations in R for storing and analyzing numerical business data such as monthly sales figures.

Sample Input	Expected Output
Enter number of months: 5	Sales Vector: 20000 22000 19000 19500 21000
Enter sales for month 1: 20000	Total Sales: 101500
Enter sales for month 2: 22000	Average Sales: 20300
Enter sales for month 3: 19000	Highest Sales: 22000
Enter sales for month 4: 19500	Lowest Sales: 19000
Enter sales for month 5: 21000	Above Average Sales: 22000 21000

Program:

```
# Monthly Sales Analysis using Vectors
```

```
# User input n and sales
```

```
n <- as.integer(readline("Enter the number of months: "))
```

```
Sales <- numeric(n)
```

```
for(i in 1:n){
```

```
  Sales[i] <- as.numeric(readline(paste("Enter Sales for",
```

```
  month, i, ", : j)"))
```

```
}
```

```
# Display vector
```

```
cat("Sales Vector:", Sales, "\n")
```

Input:

Enter number of months: 5

Enter sales for month 1: 20000

Enter sales for month 2: 22000

Enter sales for month 3: 19000

Enter sales for month 4: 19500

Enter sales for month 5: 21000

Output:

Sales vector: 20000 22000 19000 19500 21000

Total Sales: 101500

Average Sales: 20300

Highest Sales: 22000

Lowest Sales: 19000

Above Average Sales: 22000 21000

Calculations

```
total_sales <- sum(sales)
```

```
cat("Total Sales : ", total_sales, "\n")
```

```
avg_sales <- mean(sales)
```

```
cat("Average Sales : ", avg_sales, "\n")
```

```
high_sales <- max(sales)
```

```
cat("Highest Sales : ", high_sales, "\n")
```

```
lowest_sales <- min(sales)
```

```
cat("Lowest Sales : ", lowest_sales, "\n")
```

Above average sales

```
above_avg <- sales[sales > avg_sales]
```

```
cat("Above Average Sales : ", above_avg, "\n").
```

Result:

Thus, the program analyzes monthly sales data using vector operations in R. It demonstrates efficient aggregation and comparison of business data using vectors.

Input:

Enter number of households: 3
 Enter number of months: 3
 Enter electricity units:
 Household 1 month 1: 250
 Household 1 month 2: 300
 Household 1 month 3: 150
 Household 2 month 1: 321
 Household 2 month 2: 112
 Household 2 month 3: 212
 Household 3 month 1: 112
 Household 3 month 2: 142
 Household 3 month 3: 155

Output:

Electricity Consumption Matrix:
 Jan Feb Mar
 Household 1 250 300 150
 Household 2 321 112 212
 Household 3 112 142 155
 Average Consumption per Month:
 Jan Feb Mar
 220.6667 229.3333 172.3333
 Highest Consumption: 321
 Lowest Consumption: 112.

Ex: No: 1F
 Date: 24/1/26

Household Electricity Consumption Analysis Using Matrix

Problem Description:

An electricity board records monthly electricity consumption (in units) of multiple households. The data is stored in a matrix where **rows represent households** and **columns represent months**. Write an R program to analyze electricity usage by calculating row-wise total consumption, column-wise average consumption, highest usage, and lowest usage

Objective:

To demonstrate the use of matrices in R for storing and analyzing two-dimensional numerical data such as electricity consumption records.

Sample Input	Expected Output
Enter number of households: 3	Electricity Consumption Matrix:
Enter number of months: 3	Jan Feb Mar
Enter electricity units:	Household1 250 225 150
Household 1 Month 1: 250	Household2 300 321 212
Household 1 Month 2: 300	Household3 112 142 155
Household 1 Month 3: 150	Total Consumption per Household:
Household 2 Month 1: 321	Household1 Household2 Household3
Household 2 Month 2: 112	625 833 409
Household 2 Month 3: 212	Average Consumption per Month:
Household 3 Month 1: 112	Jan Feb Mar
Household 3 Month 2: 142	220.6667 229.3333 172.3333
Household 3 Month 3: 155	Highest Consumption: 321
	Lowest Consumption: 112

Program:

```
# Electricity Consumption Analysis using Matrix with Row and Column Names
# User input households, months and units
```

```
h <- as.integer(readline("Enter number of households: "))
m <- as.integer(readline("Enter number of months: "))
cons <- matrix(0, nrow = h, ncol = m)
cat("Enter electricity units: \n")
for(i in 1:h) {
  for(j in 1:m) {
    cons[i,j] <- as.numeric(
```

```
readline(paste("Household", i, "Month", j, ": "))
```

```
}
}
```

Assign row and column names

```
row.names(cons) <- paste("Household", 1:h, sep = " ")
```

```
col.names(cons) <- c("Jan", "Feb", "Mar")[1:m]
```

Display matrix

```
cat("Electricity consumption Matrix:\n")
```

```
print(cons)
```

Row-wise total

```
row_total <- rowSums(cons)
```

```
cat("Total consumption per Household:\n")
```

```
print(row_total)
```

Column-wise average

```
col_avg <- colMeans(cons)
```

```
cat("Average consumption per month:\n")
```

```
print(col_avg)
```

Maximum and minimum consumption

```
cat("Highest consumption:", max(cons), "\n")
```

```
cat("Lowest consumption:", min(cons), "\n")
```

Result:

Thus, the program analyzes household electricity consumption using matrix operations in R. It demonstrates efficient processing of two-dimensional data through row-wise and column-wise computations.

Problem Description:

A college maintains the marks of students across multiple assessments and months for academic monitoring. The marks of students over several days, assessments, and months are recorded. The college wants to store this data in a multidimensional array (Students × Days × Assessments × Months) and identify marks below a user-defined threshold for remedial action.

Objective:

To apply multidimensional array operations in R for storing and analyzing student marks across multiple days, assessments, and months, and to identify low-scoring records based on a user-defined threshold.

Sample Input	Expected Output
Enter number of students: 2 Enter number of days per assessment: 2 Enter number of assessments: 2 Enter number of months: 2 Enter minimum marks threshold: 50	<pre> --- Month 1 --- Enter marks for Assessment 1 : Enter marks for Student 1 for 2 days separated by space: 1: 56 64 Read 2 items Enter marks for Student 2 for 2 days separated by space: 1: 45 52 Read 2 items Enter marks for Assessment 2 : Enter marks for Student 1 for 2 days separated by space: 1: 75 40 Read 2 items Enter marks for Student 2 for 2 days separated by space: 1: 65 38 Read 2 items --- Month 2 --- Enter marks for Assessment 1 : Enter marks for Student 1 for 2 days separated by space: 1: 65 85 Read 2 items Enter marks for Student 2 for 2 days separated by space: 1: 45 75 Read 2 items Enter marks for Assessment 2 : Enter marks for Student 1 for 2 days separated by space: 1: 56 47 Read 2 items Enter marks for Student 2 for 2 days separated by space: 1: 50 56 Read 2 items </pre>

Input:

```

Enter number of students: 2
Enter number of days per assessment: 2
Enter number of assessments: 2
Enter number of months: 2
Enter minimum marks threshold: 50

```

Output:

```

month 1
Enter marks for Assessment 1:
Enter marks for student 1 for 2 days separated by space:
1: 56 64
Read 2 items
Enter marks for student 2 for 2 days separated by space:
1: 45 52
Read 2 items
Enter marks for Assessment 2:
Enter marks for student 1 for 2 days separated by space:
1: 75 40
Read 2 items
Enter marks for student 2 for 2 days separated by space:
1: 65 38
Read 2 items

```


marks

Enter the marks for assessment 1:

Enter the marks for student 1 for 8 days separated by space:

1: 65 85

Read 2 items:

Enter the marks for student 2 for 8 days separated by space:

2: 50 56

Read 2 items:

Marks Array (4-D):

1: Assessment 1, month 1

Student 1 Day 1 Day 2

Student 2 69 52

2: Assessment 2, month 1

Student 1 Day 1 Day 2

Student 2 75 65

Student 2 40 38

1: Assessment 1, month 2

Day 1 Day 2

Student 1 65 45

Student 2 85 75

2: Assessment 2, month 2

Day 1 Day 2

Student 1 56 50

Student 2 47 56

Low marks records (50) for month 1:

Assessment 1:

Student 1 Day 1: 45

Student 2 Day 1: 38

Marks Array (4-D):			
.. Assessment 1, Month 1			
	Day 1	Day 2	
Student 1	56	45	
Student 2	64	52	
.. Assessment 2, Month 1			
	Day 1	Day 2	
Student 1	75	65	
Student 2	40	38	
.. Assessment 1, Month 2			
	Day 1	Day 2	
Student 1	65	45	
Student 2	85	75	
.. Assessment 2, Month 2			
	Day 1	Day 2	
Student 1	56	50	
Student 2	47	56	
Low Marks Records (< 50) for month 1:			
Assessment 1:			
Student 1 Day 1:	45		
Assessment 2:			
Student 1 Day 2:	47		
Student 2 Day 1:	38		
Student 2 Day 2:	38		

Program:

User input for dimensions - students, days, assessments, months and threshold

Students ← as integer (readline("Enter number of students: "))

days ← as integer (readline("Enter number of days per assessment: "))

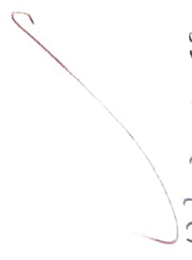
as ← as integer (readline("Enter number of assessments: "))

months ← as integer (readline("Enter number of months: "))

threshold ← as integer (readline("Enter minimum marks threshold: "))

Empty vector to store marks

marks_vector ← ()



for mark receiving (250) for month:

Assignment 1:

Student 1 day 8: 45

Assignment 2:

Student 2 day 1: 47

Input marks for each month, assessment, and student

for (m in 1:nmonths) {

for (s in 1:nstudents) {

for (a in 1:nassessments) {

val ["\n--month", m, "--\n"]

for (s in 1:nstudents) {

for (a in 1:nassessments) {

"days separated by space: \n")

values ← scan(n=days)

marks-vector ← c(marks-vector, values) } }

Create 4-D array

marks-array ← array(marks-vector, dim = c(students, days, assessments, month),

dimnames = list (

paste ("Student", 1:students),

paste ("Day", 1:days),

paste ("Assessment", 1:assessments),

paste ("Month", 1:nmonths)

),

```
# Display the multidimensional array
```

```
cat("\nMarks Array (A-D):\n")
```

```
Print(marks_array)
```

```
# Identify low marks records
```

```
for (m in 1:months){
```

```
  cat("\nLow Marks Records (< ", threshold, ")
```

```
for Month", m, ":\n")
```

```
  for (a in 1:assessments){
```

```
    cat("\nAssessment", a, ":\n")
```

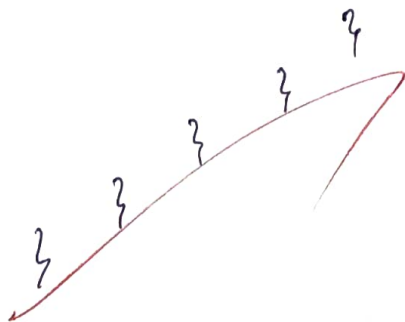
```
    for (s in 1:students){
```

```
      for (d in 1:days){
```

```
        value ← marks_array[s,d,a,m]
```

```
        if (value < threshold){
```

```
          cat("Student", s, "Day", d, ":",  
value, "\n")
```



Result:

Thus, the program successfully demonstrates the use of multidimensional arrays in R to store and analyze student marks across multiple days, assessments, and months, and to identify low-scoring records based on a user-defined threshold.

Sample Input.

Enter number of clients: 2
 Enter maximum travel days: 10
 Enter detail for client 1:
 Enter client name: Des1
 Enter itinerary ID: A123
 Enter number of Destinations: 3
 Enter destination 1 name: Des1
 Enter number of days in Des1: 6
 Enter mode of transport to Des1: Flight
 Enter destination 2 name: Des2
 Enter number of days in Des2: 3
 Enter mode of transport to Des2: Car
 Enter destination 3 name: Des3
 Enter number of days in Des3: 2
 Enter mode of transport to Des3: Car
 Enter details for client 2:
 Enter client name: Aaron
 Enter itinerary ID: A321
 Enter number of destinations: 2
 Enter destination 1 name: Des3
 Enter number of days in Des3: 2
 Enter mode of transport to Des3: Car
 Enter destination 2 name: Des4
 Enter number of days in Des4: 1
 Enter mode of transport to Des4: Car

Enter mode of transport to Des1: Flight
 Enter destination 3 name: Des5
 Enter number of days in Des5: 5
 Enter mode of transport to Des5: Car

Travel Itinerary Management Using Lists

Ex.No: 1H
 Date: 9/2/26

Problem Description:

A travel agency wants to maintain custom travel itineraries for multiple clients. Each itinerary includes:

- Client Name (string)
- Itinerary ID (string)
- Destinations (vector of city names)
- Days Spent (numeric vector, corresponding to each destination)
- Mode of Transport (vector of strings: Flight, Train, Bus)

The agency wants to:

1. Store each client's itinerary as a list, allowing heterogeneous data.
2. Calculate total travel days for each client.
3. Identify clients whose total travel exceeds a user-defined limit for special assistance.

This allows the agency to manage multiple itineraries dynamically and perform quick analyses on travel plans.

Objective:

To use lists in R to store heterogeneous travel itinerary data, calculate total travel days, and identify clients with extended trips for special planning or assistance.

Sample Input	
Enter number of clients: 2 Enter maximum recommended travel days: 10	Enter details for Client 2: Enter client name: Aaron Enter itinerary ID: A321
Enter details for Client 1: Enter client name: Des1 Enter itinerary ID: A123	Enter number of destinations: 2 Enter destination 1 name: Des4 Enter number of days in Des4: 4
Enter number of destinations: 3 Enter destination 1 name: Des1	Enter mode of transport to Des4: Flight Enter destination 2 name: Des5 Enter number of days in Des5: 5
Enter number of days in Des1: 6 Enter mode of transport to Des1: Flight	Enter mode of transport to Des5: Car
Enter destination 2 name: Des2 Enter number of days in Des2: 3	
Enter mode of transport to Des2: Car Enter destination 3 name: Des3	
Enter number of days in Des3: 2 Enter mode of transport to Des3: Car	

Output:

```

Travel Itineraries:
[[{"id": 1, "client": "A123", "days": 10, "transport": "Flight", "destinations": ["Des1", "Des2", "Des3", "Des4", "Des5"]}, {"id": 2, "client": "A321", "days": 5, "transport": "Flight", "destinations": ["Des1", "Des2", "Des3", "Des4", "Des5"]}]]
Clients with total travel days exceeding 10:
Client: Des1 | Itinerary ID: A123 | Total Days: 10
  
```

Expected Output	
Travel Itineraries:	[[{"id": 1, "client": "A123", "days": 10, "transport": "Flight", "destinations": ["Des1", "Des2", "Des3", "Des4", "Des5"]}, {"id": 2, "client": "A321", "days": 5, "transport": "Flight", "destinations": ["Des1", "Des2", "Des3", "Des4", "Des5"]}]]
Clients with total travel days exceeding 10:	Client: Des1 Itinerary ID: A123 Total Days: 10

Program:

```

# User input num_clients and max_days
num_clients = 2
max_days = 10

# Initialize list itineraries
itineraries = []
  
```


Input client itineraries

```
for (i in 1:num-clients) {  
  cat("\n Enter details for clients ", i, " : \n")  
  client-name ← readline("Enter client name : ")  
  id ← readline("Enter client itinerary ID : ")  
  n ← as.integer(readline("Enter number of destinations : "))  
  destination ← c()  
  days ← c()  
  transport ← c()  
  for (j in 1:n) {  
    dest ← readline(paste("enter destination", j, "name: "))  
    d ← as.integer(readline(paste("Enter number of days in",  
dest, " : ")))  
    t ← readline(paste("Enter mode of transport to", dest,  
": "))  
    destination ← c(destination, dest)  
    days ← c(days, d)  
    transport ← c(transport, t)  
  }  
  itineraries[i] ← list(  
    client = client-name,  
    ID = id,  
    destination = destination,  
    Days = days,  
    Transport = transport  
  )  
}
```

Display all itineraries

```
cat("\nTravel Itineraries:\n")
Print(itineraries)
```

Identify clients exceeding travel limit

```
cat("\nClients exceeding travel limit: \n")
for(i in 1:num_clients){
  total_days <- sum(itineraries[[i]]$Days)

  if (total_days > max_days){
    cat("Client: ", itineraries[[i]]$Client,
        "\n Itinerary ID: ", itineraries[[i]]$ID,
        "\n Total Days: ", total_days, "\n")
  }
}
```

Result:

Thus, the program successfully demonstrates the use of lists in R to store heterogeneous travel itinerary data, calculate total travel days per client, and identify clients requiring special assistance due to extended travel.

Subsec (4), condition)

Sample Input

```
Enter number of cars: 3
Enter details for car 1:
Enter car model: Volkswagen
Enter registration number: V123
Enter price: 8000000
Is the car available? (TRUE/FALSE): TRUE
Enter car type (Sedan/SUV/Hatchback): Sedan

Enter details for car 2:
Enter car model: Ford
Enter registration number: F123
Enter price: 1500000
Is the car available? (TRUE/FALSE): TRUE
Enter the car type (Sedan/SUV/Hatchback): SUV

Enter details for car 3:
Enter car model: BMW
Enter registration number: B123
Enter price: 3000000
Is the car available? (TRUE/FALSE): FALSE
Enter car type (Sedan/SUV/Hatchback): Hatchback
```

Ex: No: 11

Date: 10/2/26

Car Inventory Management Using Data Frame and Factor

Problem Description:

A car dealership wants to maintain an inventory of cars, including:

- Car Model (string)
- Registration Number (string)
- Price (numeric)
- Availability (logical: TRUE/FALSE)
- Car Type (categorical: Sedan, SUV, Hatchback)

The dealership wants to:

1. Store car data in a data frame.
2. Represent Car Type as a factor.
3. Search cars based on availability and type.

All data should be entered by the user dynamically.

Objective:

To use data frames and factors in R to store heterogeneous car inventory data, categorize car types, and search for cars based on availability and type.

Sample Input

```
Enter number of cars: 3
Enter details for Car 1:
Enter car model: Volkswagen
Enter registration number: V123
Enter price: 2000000
Is the car available? (TRUE/FALSE): TRUE
Enter car type (Sedan/SUV/Hatchback): Sedan

Enter details for Car 2:
Enter car model: Ford
Enter registration number: F123
Enter price: 1500000
Is the car available? (TRUE/FALSE): TRUE
Enter car type (Sedan/SUV/Hatchback): SUV

Enter details for Car 3:
Enter car model: BMW
Enter registration number: B123
Enter price: 3000000
Is the car available? (TRUE/FALSE): FALSE
Enter car type (Sedan/SUV/Hatchback): Hatchback
```

Output.

Complete Car Inventory:

	Model	RegNo	Price	Available	Type
1	Volkswagen	V123	2000000	TRUE	Sedan
2	Ford	F123	1500000	TRUE	SUV
3	BMW	F123	3000000	FALSE	Hatchback

Do you want to see available cars only? (TRUE/FALSE): TRUE

Available Cars:

	Model	RegNo	Price	Available	Type
1	Volkswagen	V123	2000000	TRUE	Sedan
2	Ford	F123	1500000	TRUE	SUV

Enter car type to search (Sedan/SUV/Hatchback): SUV

Cars of type SUV:

	Model	RegNo	Price	Available	Type
2	Ford	F123	1500000	TRUE	SUV

Expected Output

```
Complete Car Inventory:
Model RegNo Price Available Type
1 Volkswagen V123 2000000 TRUE Sedan
2 Ford F123 1500000 TRUE SUV
3 BMW B123 3000000 FALSE Hatchback

Do you want to see available cars only? (TRUE/FALSE): TRUE

Available Cars:
Model RegNo Price Available Type
1 Volkswagen V123 2000000 TRUE Sedan
2 Ford F123 1500000 TRUE SUV

Enter car type to search (Sedan/SUV/Hatchback): SUV

Cars of type SUV:
Model RegNo Price Available Type
2 Ford F123 1500000 TRUE SUV
```

Program:

User input num_cars

num_cars ← as.integer(readline("Enter number of cars: "))

Initialize empty vectors Models, RegNos, Prices, Available, CarType

Models ← c()

RegNos ← c()

Prices ← c()

Available ← c()

CarType ← c()

Input car details

for (i in 1:num_cars) {

cat("\nEnter details for car ", i, ":\n")

model ← readline("Enter car model: ")

regno ← readline("Enter registration number: ")

price ← as.numeric(readline("Enter price: "))


```

avail ← as.logical(readline("Is the car available? (TRUE/
FALSE): "))
type ← readline("Enter car type (Sedan / SUV / Hatchback): ")
Models ← c(Models, model)
RegNos ← c(RegNos, regno)
Prices ← c(Prices, price)
Available ← c(Available, avail)
} CarType ← c(CarType, type)
# Convert CarType to factor

CarType ← factor(CarType)

```

```

# Create data frame car_df

```

```

car_df ← data.frame(Models, RegNos, Prices, Available, CarType)

```

```

# Display complete car inventory

```

```

cat("\nComplete Car Inventory: \n")
Print(car_df)

```

Search cars by availability

```
check-avail ← as.logical(readline("\n Do you want to see  
available cars Only? (TRUE/FALSE): "))
```

```
if (check-avail == TRUE) {
```

```
  cat("\n Available Cars :\n")
```

```
  Print(car-df [car-df $Available == TRUE, ])
```

```
}
```

Search cars by type

```
Search-type ← readline("\n Enter car type to search  
(Sedan / SUV / Hatchback): ")
```

```
cat("\n Cars of Type", search-type, ":\n")
```

```
Print(car-df [car-df $ CarType == search-type, ])
```

Result:

Thus, the program successfully demonstrates the use of **data frames and factors in R** to store **heterogeneous car inventory data**, categorize cars by **type**, and **search cars based on availability and type**.